

feelzlike.com: full site review, smoke and security test

Reviewer: Sven | Date: 25 Sep 2026 | Build: /assets/index-myr3f-3N.js, repo commit 3c8ecd84

Scope: I tested the live site from outside without logging in, and reviewed the code in DarkHorse263/Yamanouchi-Weather-App. I made no purchases and created no checkout sessions. Stripe was only probed with rejected (unauthenticated) calls.

Verdict

The site itself is in good shape. Every smoke page loaded, and the fixes from last round are live. The Stripe server code is well built.

There are three problems to fix before purchases open:

- 1. **The paywall is only enforced in the browser.** No API route uses requireEntitlement, so anyone can pull the premium data directly from the API.
- 2. **The new elevation logic broke two groups of resorts.** Snow is now forecast at the village for Thredbo, Perisher, Selwyn and Charlotte's Pass. It is forecast at the summit for about 100 other resorts, including Whistler, Mt Buller and Coronet Peak.
- 3. **The weather feed falls over under load.** When about 200 uncached resorts were requested in roughly a minute, 260 of 648 returned 500. The supplier is Open-Meteo, whose free tier does not allow commercial use.

Smoke test

Area	Result
Home, /countries, /premium, /alerts, /compare, /about, /near-you, /legal/privacy, /legal/terms	Pass: all load with correct titles
Thredbo, Perisher, Jindabyne, Heavenly, Jackson Hole, Stowe, Holiday Valley, Niseko Town, Whistler	Pass: all APIs returned 200
Mobile (390x844)	Pass: the prompt overlap is fixed, and only the cookie banner sits above the nav
Units	Pass: Perisher defaults to °C and km/h
US/CA snow heights	Pass: Heavenly 2488, Jackson 2555, Stowe 874, Holiday Valley 488
AU/NZ/CA snow heights	Fail: Thredbo 1365 (base), Perisher 1720 (base), Mt Buller 1805 (summit), Whistler 2182 (summit), Coronet 1649 (summit)
Full weather sweep (648 ids, 3 at a time)	Fail: 387 returned 200, 260 returned 500 ("No weather data available from any source"), 1 returned 404. It recovered within about 20 seconds
/pricing	Fail: redirects to home, but the 402 upgrade link points to /pricing
/account signed out	Shows the home title. Check that it prompts sign-in

Area	Result
Unknown page, e.g. /nonexistent-page	Returns 200 with the app shell (soft 404, bad for SEO)
robots.txt, sitemap (2,829 URLs), manifest, sw.js	Pass
Leaked internal note	Fail: the Whitefish blurb still says "no confirmed dedicated official webcam URL found in research, treat as unverified"
Premium page	"at" is shown instead of "Austria" in the powder-alert region list

Security findings (ranked)

High

- Premium data is not protected on the server.** `requireEntitlement` is defined, but no route uses it.
 - `/api/elevation-forecast` (upper, mid and lower bands) returns 200 to anonymous users.
 - `/api/weather/{id}` returns all 14 days to anyone.
 - The `PremiumGate` only hides these in the UI. Once the promo ends, anyone can get premium content for free with curl or devtools.
- Forecasts can be forced to miss the cache, so the weather feed can be knocked out.** The cache key is `id@snowElevationM`, and the server accepts any value from 1 to 9000.
 - An attacker can cycle through heights so every request goes to the supplier. The per-IP limit is 120 per minute.
 - My polite sweep already triggered supplier failures, and there is no fallback to a stale cached forecast. Once the supplier rate-limits you, every user sees errors.
- Open-Meteo licensing.** The server calls `api.open-meteo.com` without an API key. Open-Meteo's [terms](#) mark commercial use as not allowed on the free tier. A [subscription](#) gives a commercial licence and a dedicated customer endpoint. A paid app needs this before charging.

Medium

- Regional prices can be gamed.** Visitors pick their billing country themselves, and it defaults to Australia. A US browser saw "AUD \$5.99" with only a "use United States" suggestion.
 - A US user who keeps AU pays about USD 3.90 with tax included, instead of USD 5.99 plus tax.
 - The server only checks that the Stripe metadata matches the chosen country. It never compares that against the billing address Stripe collects.
- The OpenWeatherMap key is public.** `VITE_OWM_API_KEY` is embedded in the site's JavaScript, so anyone can use your quota. Proxy those calls through the API server and rotate the key. The Clerk `pk_live` key and the Sentry DSN are meant to be public, so they are fine.

6. **CORS trusts any *.replit.app, *.replit.dev or *.repl.co site, with credentials, in production.** It is confirmed live: Origin: `https://evil.replit.app` is reflected back with `allow-credentials: true`. `SameSite=Lax` cookies limit the damage, but in production only the feelzlike origins should be allowed.
7. **/api/billing/status has no rate limit.** Each call runs `billingReady()`, which makes about 10 or more Stripe API calls. One signed-in user could burn through your Stripe rate limit. Also, `paidEntitlement` will make 2 Stripe calls on every gated request once gating is switched on.
8. **The HTML pages send no security headers except HSTS.** There is no CSP, X-Frame-Options or frame-ancestors, nosniff, Referrer-Policy or Permissions-Policy, so the account and billing pages can be framed (clickjacking). The API does send nosniff and a referrer policy.

Low

9. **An unknown CORS origin gets HTTP 500 instead of a clean response without CORS headers.**
10. **The Stripe webhook returns 503 for a bad signature (it should be 400).** Missing signatures correctly return 400.
11. **HSTS is sent twice on API responses with different values (1 year and 2 years).**
12. **The promo end date uses the server clock (UTC).** The server lets promo access run until about 11am on 1 Jan AEDT, while the browser ends it at midnight local time. Set the end time explicitly, e.g. `2026-12-31T23:59:59+11:00`.
13. **The GAESA cookie has no Secure or HttpOnly flag visible.** It comes from the hosting layer.

Passed

- `/api/account` and `/api/billing/status` return 401 without login.
- `/api/admin*` returns 403.
- Anonymous checkout returns 403.
- `/.env`, `/.git/config` and the `.js.map` source maps just return the app shell, so nothing is exposed.
- No Stripe secret or webhook keys are in the JavaScript.

Billing review

The Stripe server code is strong:

- Webhook signatures are verified.
- Price IDs live only on the server and are checked against the expected amounts.
- Customer ownership is checked, and idempotency keys and advisory locks are used.
- Duplicate events are ignored, and subscriptions are re-fetched from Stripe.
- Automatic tax is on and a billing address is required.

- POST requests are checked against the site origin, and the Stripe portal only accepts an empty body.

The premium page currently says "purchases not yet open".

One thing to check before launch: my POST sent `Origin: https://feelzlike.com` and got `BILLING_ORIGIN_REQUIRED`, and so did `www.feelzlike.com`. The auth check runs after the origin check, so an origin mismatch cannot be ruled out. Make sure `BILLING_ORIGIN` exactly matches the address users actually browse on.

Business point: "regional pricing" is currently the same number in each currency. AUD 5.99 including tax, USD 5.99 plus tax and EUR 5.99 including tax are very different real prices. Canada, Japan, NZ and the UK are all billed in AUD. Consider local prices (CAD, JPY, NZD, GBP) or Stripe Adaptive Pricing.

Notes for Replit (paste in order)

1. **Enforce premium on the server.** Wrap every premium API in `requireEntitlement(...)`:
`/api/elevation-forecast`, days 8 to 14 of `/api/weather/{id}` (trim to 7 days for free users), and any carpark, disruption or offline-pack endpoints. Keep the promo behaviour where any signed-in user passes. Add tests that anonymous calls get 401, and that signed-in users with no subscription get 402 after the promo.
2. **Fix the elevation regression in `snowForecastElevation` callers:**
 - (a) In `regions/snowy-mountains.ts`, `elevationM` is the summit. Add `summitElevationM = 2037` (Thredbo), 2054 (Perisher), 1614 (Selwyn) and 1837 (Charlotte's Pass), or have `LocationDetail` use `elevationM` as the summit when it is above the base.
 - (b) For the roughly 102 authored mountains that have only `elevationM` (61 US, 24 CA, 10 AU, 7 NZ, e.g. whistler-mountain, mt-buller, coronet-peak), `elevationM` is the summit. Add their `baseElevationM` so the midpoint is used, rather than falling back to the summit.
 - (c) Add a unit test so AU, NZ and CA flagship resorts request mid-mountain heights.
 - Expected heights: Thredbo about 1701, Perisher about 1887, Whistler about 1432 (base 675), Mt Buller about 1590 (base 1375).
3. **Stop cache-busting.** Ignore the client's `snowElevationM`, or snap it to the server's configured mid height for that location. Cache by location id only.
4. **Serve stale on failure.** If Open-Meteo or BOM fails, return the last good forecast (up to 6 to 12 hours old) with a `stale:true` flag, rather than a 500. Also add a background warm-up so popular resorts are always cached.
5. **Buy an Open-Meteo commercial plan** before billing opens. Switch to `customer-api.open-meteo.com` with an `OPEN_METEO_API_KEY` secret.
6. **Stop regional price gaming.** In the Stripe webhook, read the country from the subscription customer's address, or from the checkout session's `customer_details.address.country`. If `currencyForCountry(country)` does not match the charged currency, flag it and do not grant access, or cancel and refund.

- Better still, default the country picker to the browser's region.
 - Option: let Stripe pick the currency from the billing address.
7. **Protect the OWM key.** Remove `VITE_OWM_API_KEY` from the frontend, proxy those calls via `/api`, and rotate the key in OpenWeatherMap.
 8. **Tighten CORS.** In production, only allow `https://feelzlike.com` and `https://www.feelzlike.com`. Allow the `replit.app`, `replit.dev`, `repl.co` and localhost patterns only when `NODE_ENV !== "production"`. For unknown origins, return no CORS headers instead of throwing a 500.
 9. **Billing performance.**
 - Add a rate limiter to `GET /billing/status` (e.g. 30 per minute).
 - Cache the `billingReady()` result for 5 minutes.
 - Cache `paidEntitlement` per user for 60 seconds, and clear it from the webhook.
 10. **Confirm** `BILLING_ORIGIN` exactly equals the origin browsers send (currently `https://feelzlike.com`, with no trailing slash). Test one signed-in checkout in Stripe test mode.
 11. **HTML security headers** for the static site. Add:
 - `X-Frame-Options: DENY` (or `CSP frame-ancestors 'none'`)
 - `X-Content-Type-Options: nosniff`
 - `Referrer-Policy: strict-origin-when-cross-origin`
 - `Permissions-Policy: geolocation=(self), camera=(), microphone=()`
 - A report-only CSP to start, covering Clerk, Stripe, Sentry, GA and the tile hosts
 12. **Webhook:** return 400 (not 503) when `constructEvent` fails a signature check. Keep 503 for genuine processing errors, and alert on those via Sentry.
 13. **Set explicit promo boundaries** with an offset: `2026-06-01T00:00:00+10:00` to `2026-12-31T23:59:59+11:00`. Use them in both the server and the client.
 14. **Add a /pricing route** that redirects to `/premium`, or change the 402 `upgradeUrl` to `/premium`.
 15. **Remove the leaked Whitefish text** ("no confirmed dedicated official webcam URL found in research, treat as unverified"). Add a build check that fails on "treat as unverified", "per research" or a warning emoji in blurbs.
 16. **Show "Austria"** instead of "at" in the powder-alert region list.
 17. **Soft 404s:** unknown routes should render a 404 page and add `noindex`. Serve a real 404 status for asset-like paths (`*.map`, `/.env`, `/.git/*`).
 18. **Remove the duplicate HSTS header** on API responses (keep one value).